

Wie kommt man auf diese Lösung? (Lösungsstrategie)

Um diese Aufgabe zu lösen, muss man das Problem in vier Teilschritte zerlegen:

Methodensignatur definieren (Generics):

- Die Aufgabe verlangt eine generische Methode.
- **Einschränkung:** Die Elemente müssen Zahlen sein. In Java ist die Oberklasse für alle Zahlen `Number`.
- **Syntax:** `<T extends Number>`.
- **Rückgabotyp:** Es soll wieder eine `Collection` vom selben Typ `T` zurückgegeben werden.
- **Ergebnis:** `static <T extends Number> Collection<T> shuffleCollection(Collection<T> collection).`

Datenstruktur wählen (List vs. Collection):

- Der Parameter ist eine `Collection`. Eine `Collection` hat keinen Index (kein `get(i)`). Um Elemente zu vertauschen (Shufflen), benötigen wir aber Zugriff über Indizes.
- **Lösung:** Wir müssen die `Collection` in eine `List` (z. B. `LinkedList` oder `ArrayList`) umwandeln.
- **Wichtig:** Wir sollen eine *neue* `Collection` zurückgeben. Daher erzeugen wir eine Kopie: `new LinkedList<>(collection).`

Algorithmus wählen (Shuffle):

- Da `Collections.shuffle` verboten ist, müssen wir selbst mischen.
- **Logik:** Man geht die Liste von hinten nach vorne durch. Für jedes Element an Position `i` wählt man eine zufällige Position `j` zwischen 0 und `i` und tauscht die beiden.

Rückgabe (Immutability):

- Die Aufgabe verlangt eine "immutable collection" (unveränderbar).
- Java bietet hierfür Wrapper-Methoden an: `Collections.unmodifiableList(...).`

```

package org.example;

import java.util.*;

// Die Klasse ist 'final', typisch für Utility-Klassen, damit niemand davon erben kann.
final class ShuffleUtil {

    // Ein statischer Zufallsgenerator.
    // 'static': Wird einmalig für die ganze Klasse erstellt (spart Speicher im Vergleich zu 'pro Methodenaufruf').
    // 'final': Die Referenz auf das Random-Objekt ändert sich nicht mehr.
    private static final Random RANDOM = new Random();

    // Die Hauptmethode zum Mischen.
    // '<T extends Number>': Wir definieren einen Typ T. Dieser MUSS eine Zahl sein (Integer, Double, etc.).
    // 'Collection<T>': Wir nehmen eine beliebige Sammlung (Set, List, Queue) von Zahlen entgegen.
    static <T extends Number> Collection<T> shuffleCollection(Collection<T> collection) {

        // 1. Kopie erstellen & Typ umwandeln.
        // Der Input ist nur eine 'Collection' (hat keinen Index). Zum Mischen brauchen wir aber Positionen (0, 1, 2...).
        // Deshalb kopieren wir alles in eine 'LinkedList' (eine Implementierung von List).
        // Nebeneffekt: Das Original 'collection' bleibt unberührt (wichtig, da wir eine NEUE Collection zurückgeben sollen).
        List<T> result = new LinkedList<>(collection);

        // 2. Der Misch-Algorithmus (Fisher-Yates Shuffle).
        // Wir laufen von hinten (letztes Element) nach vorne durch die Liste.
        for (int i = result.size() - 1; i > 0; i--) {

            // Wähle einen zufälligen Index 'j'.
            // nextInt(bound) liefert eine Zahl von 0 bis bound-1.
            // Durch 'i + 1' erhalten wir eine Zufallszahl zwischen 0 und i (inklusive).
            // Das heißt: Wir tauschen das Element an Stelle 'i' mit irgendeinem davor (oder sich selbst).
            int j = RANDOM.nextInt(i + 1);

            // Führe den Tausch durch.
            swap(result, i, j);
        }

        // 3. Unveränderbar machen (Immutable).
        // Die Aufgabe verlangt eine immutable Collection.
        // Dieser Wrapper verhindert Schreibzugriffe (add, remove) von außen -> wirft UnsupportedOperationException.
        return Collections.unmodifiableList(result);
    }

    // Eine generische Hilfsmethode zum Tauschen von zwei Elementen in einer Liste.
    // Hier reicht '<T>' ohne 'extends Number', da Tauschen für alle Objekttypen gleich funktioniert.
    // 'private', da diese Methode nur intern von shuffleCollection genutzt wird.
    private static <T> void swap(List<T> list, int i, int j) {
        // Klassischer Dreieckstausch:
        // 1. Wert von i in temporärer Variable sichern.
        T temp = list.get(i);

        // 2. Wert von j an die Stelle von i schreiben.
        list.set(i, list.get(j));

        // 3. Den gesicherten Wert (temp) an die Stelle von j schreiben.
        list.set(j, temp);
    }
}

```

```

package org.example;

import org.junit.jupiter.api.Test;
import java.util.Collection;
import java.util.List;

// Importiert alle statischen Assertions (assertEquals, assertTrue, assertThrows),
// damit wir sie direkt ohne "Assertions." davor aufrufen können.
import static org.junit.jupiter.api.Assertions.*;

class ShuffleUtilTest {

    // Test 1: Funktionalität & Datenintegrität
    // Wir prüfen: Sind nach dem Mischen noch alle Zahlen da? Hat sich die Anzahl geändert?
    @Test
    void shuffleIntegersPreservesElements() {
        // 1. Arrange (Vorbereitung)
        List<Integer> input = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        // 2. Act (Ausführung)
        Collection<Integer> result = ShuffleUtil.shuffleCollection(input);

        // 3. Assert (Überprüfung)

        // Prüfung A: Die Größe muss gleich geblieben sein (keine Elemente verloren oder
        // verdoppelt).
        assertEquals(input.size(), result.size(), "Die Größe der Liste sollte sich nicht
        ändern");

        // Prüfung B: Alle Elemente der Ursprungsliste müssen im Ergebnis enthalten sein.
        // .containsAll() prüft, ob die Menge 'input' komplett in 'result' vorkommt.
        assertTrue(result.containsAll(input), "Es müssen alle ursprünglichen Elemente
        vorhanden sein");

        // Hinweis: Wir testen hier NICHT explizit, ob die Reihenfolge anders ist.
        // Warum? Zufall ist schwer zu testen. Es besteht eine (winzige) Chance,
        // dass der Zufall die gleiche Reihenfolge generiert. Ein Test, der dann fehlschlägt,
        // wäre "flaky" (unzuverlässig).
    }

    // Test 2: Generics
    // Wir prüfen: Funktioniert die Methode auch mit Doubles, wie es '<T extends Number>'
    // verspricht?
    @Test
    void shuffleWorksWithDoubles() {
        // 1. Arrange
        List<Double> input = List.of(1.5, 2.5, 3.5);

        // 2. Act
        Collection<Double> result = ShuffleUtil.shuffleCollection(input);

        // 3. Assert
        assertEquals(input.size(), result.size());
        assertTrue(result.containsAll(input));
    }

    // Test 3: Immutability (Unveränderbarkeit)
    // Wir prüfen: Wirft die Methode wirklich einen Fehler, wenn man versucht, das Ergebnis zu
    // ändern?
    // Das ersetzt den manuellen try-catch-Block aus der main-Methode.
    @Test
    void resultIsImmutable() {
        // 1. Arrange
        List<Integer> input = List.of(1, 2, 3);
        Collection<Integer> result = ShuffleUtil.shuffleCollection(input);

        // 2. Act & Assert kombiniert
        // assertThrows prüft: "Wenn ich den Code im Lambda-Ausdruck () -> { ... } ausführe,
        // dann MUSS die angegebene Exception (UnsupportedOperationException.class) fliegen."
        assertThrows(UnsupportedOperationException.class, () -> {
            // Dieser Befehl ist verboten, da die Liste "unmodifiable" sein soll.
            result.add(99);
        });
    }
}

```